Dokumentacja implementacyjna

Mateusz Kamieniecki
Jan Rękas

27 marca 2026

Spis treści

1 main.c **2**

2 fileio.c oraz fileio.h **3**
2.1 fileio.h 3
2.2 fileio.c 3
2.2.1 find_max_vertex_id 3
2.2.2 load_graph_from_file 3
2.2.3 save_layout_human 4
2.2.4 save_layout_binary 4

3 graph.c oraz graph.h **4**
3.1 Graph.h 4
3.1.1 struct Edge 4
3.1.2 struct Vector 5
3.1.3 struct Graph 5
3.2 graph.c 5
3.2.1 vector_init 5
3.2.2 vector_push_back 5
3.2.3 vector_free 5
3.2.4 create_graph 6
3.2.5 add_edge 6
3.2.6 free_graph 6

4 node.h **6**
4.0.1 struct Node 6
4.0.2 struct GraphLayout 6

5 tutte_embedding.c oraz tutte_embedding.h **7**
5.1 tutte_embedding.c 7
5.1.1 tutte_embedding 7
5.1.2 solve_gauss 7
5.1.3 find_triangle 7
5.1.4 fix_triangle 7

6 spectral_layout.c oraz spectral_layout.h **7**
6.1 spectral_layout.c 7
6.1.1 compute_spectral_layout 8
6.1.2 compute_degrees 8
6.1.3 laplacian_multiply 8
6.1.4 find_eigenvector 8
6.1.5 orthogonalize 8
6.1.6 normalize_vector 8
6.1.7 dot_product 9

1 main.c

W funkcji main mamy wczytywanie argumentów do odpalania programu przy użyciu getopt(). Argumenty mają domyślnie wartości:

- -i: input.txt
- -o: . (domyslny katalog gdzie znajduje się program)
- -n: output
- -a: 1

Jak żaden z argumentów *-h* oraz *-b* nie jest podany to program automatycznie uruchamia się z flagą *-b* włączoną i informuje użytkownika. W funkcji main są też odpalane funkcje do wczytywania danych z pliku, wypisywania danych do pliku/plików, funkcje tworzące graf z plików oraz funkcje do przeprowadzenia algorytmów.

2 fileio.c oraz fileio.h

Pliki `fileio.c` i `fileio.h` odpowiadają za wczytywanie grafu z pliku tekstowego oraz zapisywanie wyników obliczeń do pliku wyjściowego w formacie tekstowym lub binarnym.

2.1 fileio.h

Nagłówek deklaruje cztery funkcje:

```
Graph* load_graph_from_file(const char *filepath);
int    find_max_vertex_id(FILE *file);
void   save_layout_human (const char *filepath,
   GraphLayout *layout);
void   save_layout_binary(const char *filepath,
   GraphLayout *layout);
```

2.2 fileio.c

2.2.1 find_max_vertex_id

Pomocnicza funkcja przechodzi przez cały plik metodą `fscanf`, wczytując kolejne linie w formacie `<nazwa> <from> <to> <waga>` i wyznacza najwyższy identyfikator wierzchołka spośród wszystkich wartości `from` i `to`. Po zakończeniu pętli wywołuje `rewind(file)`, aby wskaźnik pliku powrócił na jego początek — jest to niezbędne, ponieważ plik jest następnie wczytywany po raz drugi przez `load_graph_from_file`. Funkcja zwraca `0` gdy plik jest pusty lub równy `NULL`.

2.2.2 load_graph_from_file

Funkcja otwiera plik o podanej ścieżce. W przypadku błędu wypisuje komunikat na `stderr` i zwraca `NULL`.
Następnie wywołuje `find_max_vertex_id`, aby ustalić wymaganą wielkość struktury grafu. Jeżeli wynik wynosi `0` (pusty plik), zwalnia zasoby i zwraca `NULL`.
Graf tworzony jest z `vertices_amount + 1` wierzchołkami — indeks `0` pozostaje nieużywany, dzięki czemu identyfikatory wierzchołków z pliku wejściowego (zaczynające się od `1`) odpowiadają wprost indeksom tablicy `adj`.

Pętla `fscanf` wczytuje kolejne krawędzie i dodaje je do grafu wywołaniem `add_edge`.

2.2.3 save_layout_human

Funkcja otwiera plik do zapisu. W przypadku błędu wypisuje komunikat na `stderr` i kończy działanie.
Dla każdego węzła w strukturze `GraphLayout` zapisuje wiersz w formacie:

```
<id> <x> <y>
```

Po zakończeniu wypisuje potwierdzenie na standardowe wyjście.

2.2.4 save_layout_binary

Funkcja save_layout_binary zapisuje wyniki do pliku binarnego. Dla każdego wierzchołka zapisuje kolejno jego identyfikator jako `int` (4 bajty), współrzędną x jako `double` (8 bajtów) oraz współrzędną y jako `double` (8 bajtów), co daje łącznie 20 bajtów na wierzchołek.

3 graph.c oraz graph.h

3.1 Graph.h

Graph.h udostępnia następujące structy:

3.1.1 struct Edge

```c
typedef struct Edge {
    int to;
    double weight;
    char *name;
} Edge;
```

Struct Edge reprezentuje krawędź w grafie. Zawiera ona informacje o wierzchołku docelowym, wadze krawędzi oraz nazwie krawędzi.

3.1.2 struct Vector

```
typedef struct Vector {
    Edge *data;
    int size;
    int capacity;
} Vector;
```

Struct Vector reprezentuje dynamiczną tablicę krawędzi. Zawiera ona wskaźnik na dane, aktualny rozmiar oraz pojemność tablicy.

3.1.3 struct Graph

```
typedef struct Graph {
    int verticies_num;
    Vector *adj;
} Graph;
```

Struct Graph reprezentuje graf nieskierowany. Zawiera ona liczbę wierzchołków oraz wskaźnik na tablicę wektorów, gdzie każdy wektor reprezentuje listę sąsiedztwa dla danego wierzchołka.

3.2 graph.c

Graph.c udostępnia następujące funkcje:

3.2.1 vector_init

Funkcja vector_init inicjalizuje wektor, ustawiając jego rozmiar na 0, pojemność na 4 oraz alokując pamięć dla danych.

3.2.2 vector_push_back

Funkcja vector_push_back dodaje krawędź do wektora. Jeśli rozmiar wektora osiągnie jego pojemność, funkcja podwaja pojemność i realokuje pamięć dla danych.

3.2.3 vector_free

Funkcja vector_free zwalnia pamięć zajmowaną przez dane wektora.

3.2.4 create_graph

Funkcja create_graph tworzy graf o określonej liczbie wierzchołków, alokując pamięć dla tablicy wektorów i inicjalizując każdy wektor.

3.2.5 add_edge

Funkcja add_edge dodaje krawędź do grafu, aktualizując listę sąsiedztwa dla obu wierzchołków (ponieważ graf jest nieskierowany).

3.2.6 free_graph

Funkcja free_graph zwalnia pamięć zajmowaną przez graf, w tym pamięć dla list sąsiedztwa i samego grafu.

4 node.h

Node.h udostępnia dwa struktury:

4.0.1 struct Node

```
typedef struct Node {
    int id;
    double x;
    double y;
} Node;
```

Struct `Node` reprezentuje wierzchołek w grafie, zawierając informacje o jego współrzędnych (x, y) oraz unikalnym identyfikatorze `id`.

4.0.2 struct GraphLayout

```
typedef struct {
    int count;
    Node *nodes;
} GraphLayout;
```

Struct `GraphLayout` przechowuje końcowe wyniki obliczeń — tablicę `nodes` zawierającą `count` wierzchołków wraz z wyznaczonymi współrzędnymi. Jest

to typ przekazywany do funkcji zapisu w `fileio.c` i zwracany przez funkcje algorytmów.

5 tutte_embedding.c oraz tutte_embedding.h

5.1 tutte_embedding.c

Tutte_embedding.c udostępnia następujące funkcje:

5.1.1 tutte_embedding

Funkcja tutte_embedding implementuje algorytm Tuttego do osadzania grafu w płaszczyźnie.

5.1.2 solve_gauss

Funkcja solve_gauss rozwiązuje układ równań liniowych metodą eliminacji Gaussa, która jest wykorzystywana w algorytmie Tuttego do obliczania współrzędnych wierzchołków.

5.1.3 find_triangle

Funkcja find_triangle znajduje trójkąt w grafie, który jest potrzebny do algorytmu Tuttego, aby ustalić pozycje trzech wierzchołków, które będą umieszczone na zewnątrz.

5.1.4 fix_triangle

Funkcja fix_triangle poprawia pozycje trzech wierzchołków, które zostały znalezione przez funkcję find_triangle, aby zapewnić, że są one umieszczone na zewnątrz i tworzą trójkąt równoboczny, który jest potrzebny do algorytmu Tuttego.

6 spectral_layout.c oraz spectral_layout.h

6.1 spectral_layout.c

Spectral_layout.c udostępnia następujące funkcje:

6.1.1 compute_spectral_layout

Funkcja compute_spectral_layout implementuje algorytm rozmieszczenia spektralnego. Współrzędne x wierzchołków wyznaczane są na podstawie wektora Fiedlera (drugi wektor własny macierzy Laplace'a, odpowiadający najmniejszej niezerowej wartości własnej λ_2), a współrzędne y na podstawie trzeciego wektora własnego (λ_3). Funkcja wywołuje kolejno funkcje pomocnicze i zwraca wypełnioną strukturę GraphLayout.

6.1.2 compute_degrees

Funkcja compute_degrees oblicza stopień każdego wierzchołka grafu, iterując po jego liście sąsiedztwa i zapisując wyniki do tablicy degrees, która jest wymagana przez pozostałe funkcje algorytmu.

6.1.3 laplacian_multiply

Funkcja laplacian_multiply oblicza iloczyn macierzy Laplace'a i wektora bez jawnego budowania macierzy — korzysta bezpośrednio z listy sąsiedztwa grafu oraz tablicy stopni wierzchołków, co pozwala uniknąć alokacji tablicy rozmiaru n^2.

6.1.4 find_eigenvector

Funkcja find_eigenvector wyznacza kolejny wektor własny macierzy Laplace'a metodą potęgową z deflacją. Przyjmuje tablicę wcześniej znalezionych wektorów własnych i ortogonalizuje względem nich wynik, aby uzyskać następny w kolejności wektor własny.

6.1.5 orthogonalize

Funkcja orthogonalize przeprowadza ortogonalizację Grama–Schmidta danego wektora względem przekazanej bazy wektorów, co jest wykorzystywane w trakcie metody potęgowej do eliminacji już znalezionych wektorów własnych.

6.1.6 normalize_vector

Funkcja normalize_vector normalizuje wektor do długości jednostkowej, dzieląc każdy jego element przez normę euklidesową. Jest wywoływana po każdej

iteracji metody potęgowej.

6.1.7 dot_product

Funkcja dot_product oblicza iloczyn skalarny dwóch wektorów. Jest funkcją pomocniczą wykorzystywaną przez orthogonalize oraz find_eigenvector do sprawdzania zbieżności.